

Chapter 7. Metalearners

Just to recap, in [Part III](#) you're focusing on treatment effect heterogeneity, that is, identifying how units respond differently to the treatment. In this framework, you want to estimate:

$$\tau_i(x) = E[Y_i(1) - Y_i(0)|X] = E[\tau_i|X]$$

or, $E[\delta Y_i(t)|X]$ in the continuous case. In other words, you want to know how sensitive the units are to the treatment. This is super useful in the case where you can't treat everyone and need to do some prioritization of the treatment; for example, when you want to give discounts but have a limited budget. Or when the treatment effect is positive for some units but negative for others.

Previously, you saw how you could use regression with interaction terms to get conditional average treatment effect (CATE) estimates. Now, it's time to throw some machine learning algorithms into the mix.

Metalearners are an effortless way to leverage off-the-shelf predictive machine learning algorithms for approximating treatment effects. They can be used to estimate the ATE, but, in general, they are mostly used for CATE estimation, since they can deal pretty well with high-dimensional data.

Metalearners serve to recycle predictive models for causal

inference. All predictive models, such as linear regression, boosted decision trees, neural networks, or Gaussian processes, can be repurposed for causal inference using the approaches described in this chapter. Therefore, the success of the metalearner is highly contingent on the machine learning technique it uses under the hood. Oftentimes you'll just have to try out many different things and see what works best.

Metalearners for Discrete Treatments

Suppose you work for the marketing team at an online retailer. Your goal is to figure out which customers are receptive to a marketing email. You know this email has the potential of making customers spend more, but you also know that some customers don't really like to receive marketing emails. To solve this problem, you intend to estimate the conditional average treatment effect of the email on customers' future purchase volume. That way, your team can use this estimate to decide who to send it to.

As with most business applications, you have a bunch of historical data where you've sent marketing emails to customers. You can use that abundant data to fit your CATE model. On top of that, you also have a few data points from an experiment where the marketing email was randomized.

You plan on using this precious data only for evaluating your model, as you have so little of it:

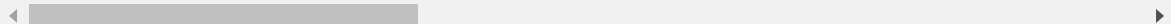
```
In [1]: import pandas as pd
import numpy as np

data_biased = pd.read_csv("./data/email_b...
data_rnd = pd.read_csv("./data/email_rr...

print(len(data_biased), len(data_rnd))
data_rnd.head()
```

```
Out[1]: 300000 10000
```

	mkt_email	next_mnth_pv	age
0	0	244.26	61.0
1	0	29.67	36.0
2	0	11.73	64.0
3	0	41.41	74.0
4	0	447.89	59.0



Both random and observational data have the exact same columns. The treatment variable is `mkt_email` and the outcome you care about is the purchase volume 1 month after receiving the email—`next_mnth_pv`. In addition to these columns, the data also contains a bunch of covariates such as customer's age, time since first purchase on your website (tenure), and also a bunch of data on how much they bought in each category. These covariates will dictate the treatment heterogeneity you plan on fitting.

To streamline the development of your CATE models, you can create variables to store the treatment, outcome, and covariates, as well as the training and testing set. Once you

have all of that, constructing pretty much all the metalearners will be straightforward:

```
In [2]: y = "next_mnth_pv"
        T = "mkt_email"
        X = list(data_rnd.drop(columns=[y, T]))

        train, test = data_biased, data_rnd
```

Now that you have everything set, let's see our first metalearner.

CAUSAL INFERENCE LIBRARIES

All of the following metalearners are implemented in most causal inference packages. However, since they are very simple to code, I'll not rely on external libraries, but instead teach you how to build them from the ground up. Also, at the time of this writing, all the causal inference packages are in their early stage, making it hard to predict which one will attain dominance in the industry. This doesn't mean you shouldn't check them out for yourself, of course. Two that I particularly like are Microsoft's `econml` and Uber's `causalml`.

T-Learner

If you have a categorical treatment, the first learner you should try is the T-learner. It is pretty straightforward and I'm guessing it is something you thought about already. It fits one outcome model $\mu_t(x)$ for every treatment in order to estimate the potential outcome Y_t . In the binary case, there are only two models that you need to estimate (hence the name T):

$$\mu_0(x) = E[Y|T = 0, X]$$

$$\mu_1(x) = E[Y|T = 1, X]$$

Once you have those models, you can make counterfactual predictions for each treatment and get the CATE as follows:

$$\hat{\tau}(x)_i = \hat{\mu}_1(X_i) - \hat{\mu}_0(X_i)$$

Figure 7-1 shows a diagram of this learner.

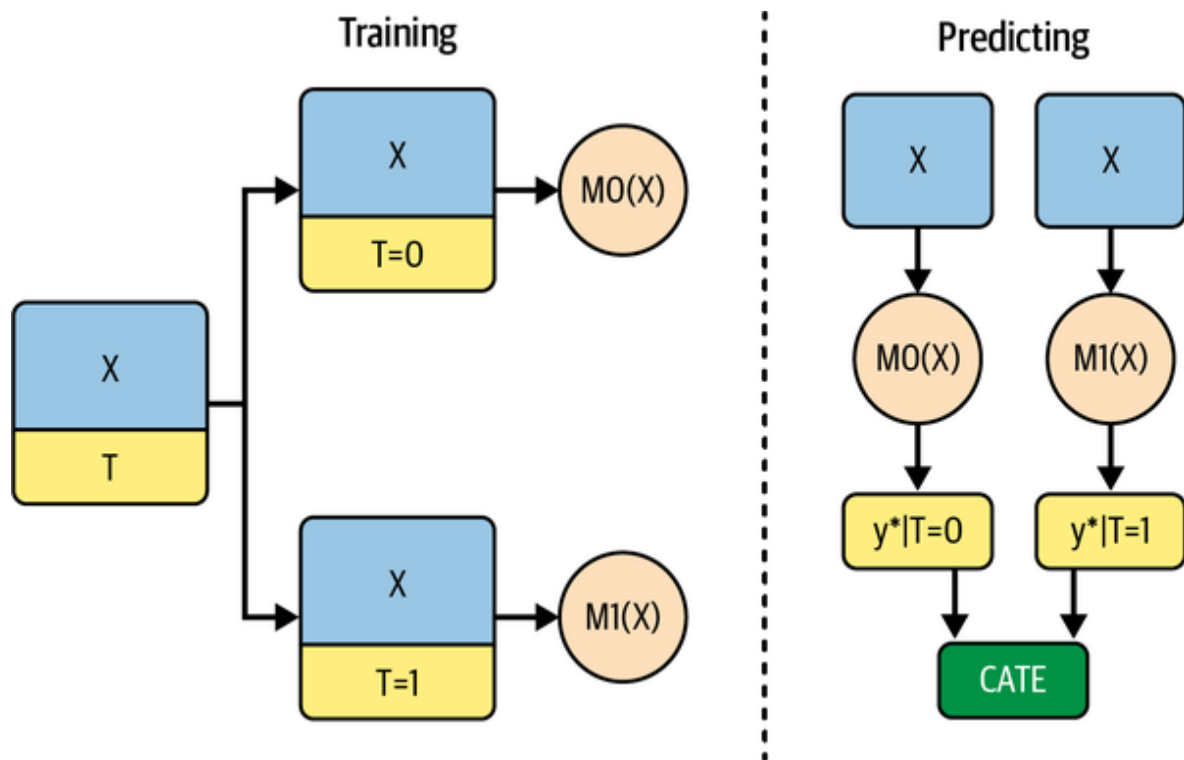


Figure 7-1. A T-learner trains an ML model on $T = 1$ and another at $T = 0$; at prediction time, it uses both models to estimate the difference between treatment and control

To code it, I'll use boosted regression trees for the outcome models. Specifically, I'll use `LGBMRegressor`, which is a very popular regression model. I'm also using the default parameters, but feel free to optimize this if you wish too:

```
In [3]: from lightgbm import LGBMRegressor

        np.random.seed(123)

        m0 = LGBMRegressor()
        m1 = LGBMRegressor()

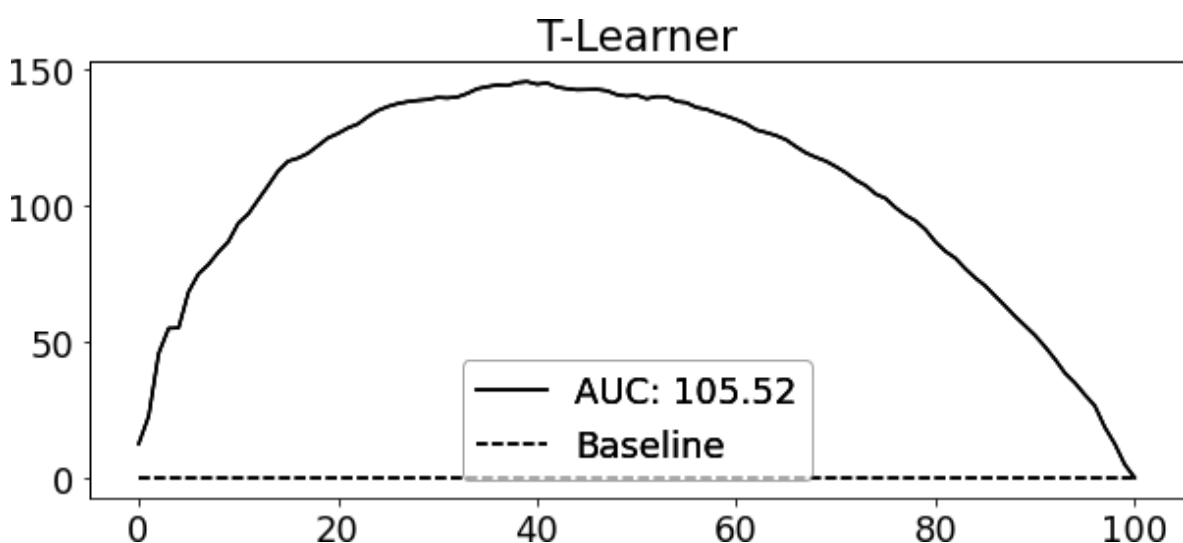
        m0.fit(train.query(f"{T}==0")[X], train
```

```
m1.fit(train.query(f"{T}==1")[X], train
```

Now that I have the two models, making CATE predictions on the test set is pretty easy:

```
In [4]: t_learner_cate_test = test.assign(  
        cate=m1.predict(test[X]) - m0.predict(  
        )
```

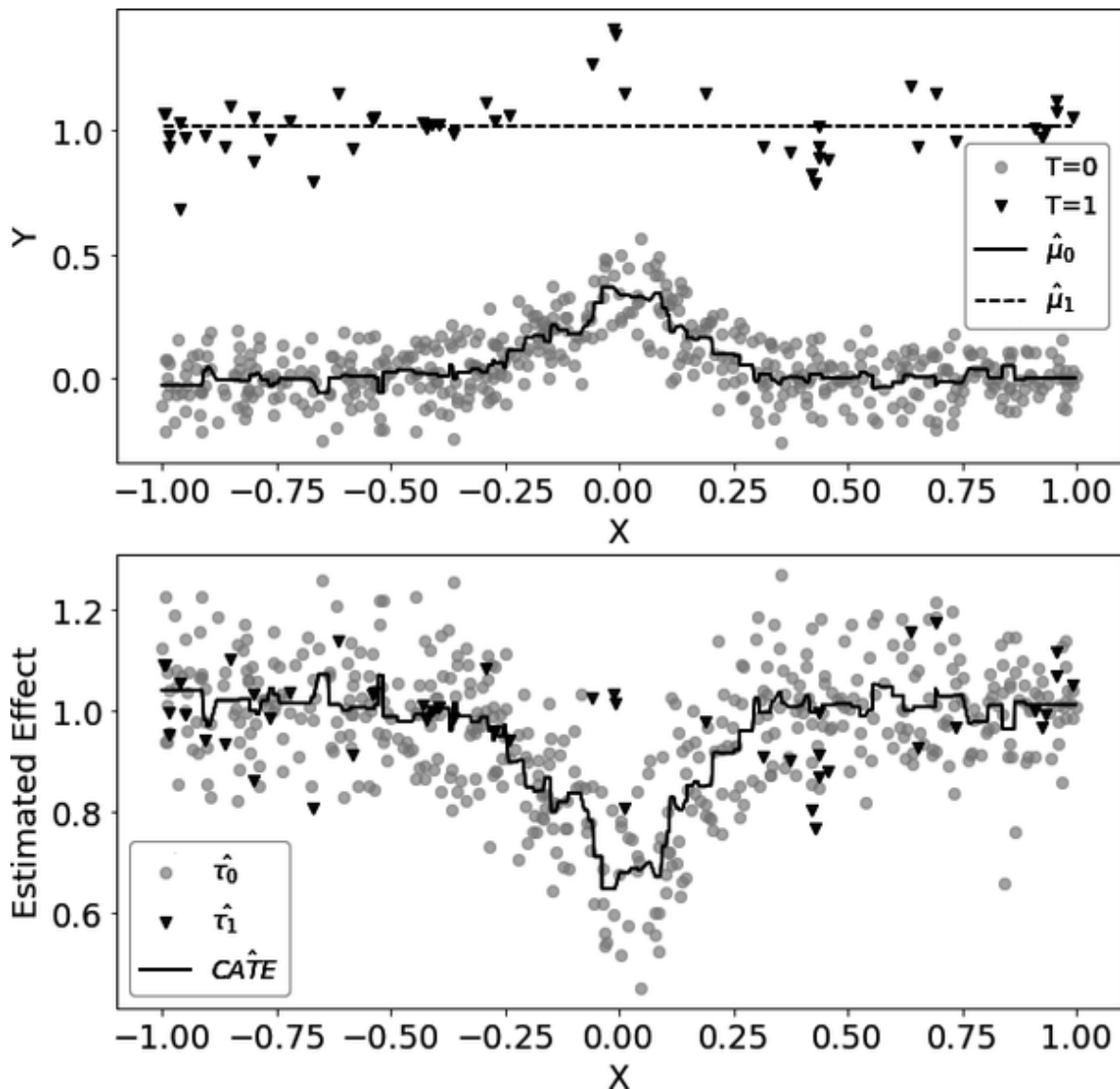
To evaluate this model, I'm using the relative cumulative gain curve and the area under that curve, both concepts that you learned in [Chapter 6](#). Recall that this evaluation method only cares if you sort customers correctly, from the one with the highest treatment effect to the one with the lowest:



The T-learner works just fine in this dataset. It looks like it can produce a pretty good ordering of customers by CATE, as you can see by the curved cumulative gain curve.

In general, the T-learner tends to be a reasonable first choice, mainly owing to its simplicity. But it has a potential issue that might manifest depending on the situation: it is prone to regularization bias.

Consider a situation where you have lots of data for the untreated and very little data for the treated. This is pretty common in many applications where the treatment is expensive. Now suppose you have some nonlinearity in the outcome Y , but the *treatment effect is constant*. This is depicted on the first plot in the following image:



If the data looks like this, with very few treated observations compared to untreated ones, there's a good chance the $\hat{\mu}_1$ model will end up being simple to avoid overfitting. In contrast, $\hat{\mu}_0$ will be more complicated, but that's OK because the abundance of data prevents overfitting. Importantly, this can happen even if you use the same hyperparameters for both models. For instance, to generate the preceding figures, I used an LGBM Regressor with

`min_child_samples=25` and everything else set to the default. A lot of ML algorithms self-regularize when dealing with fewer data points, as is the case of `min_child_samples`. It forces the tree in the LGBM to have at least 25 samples in each leaf node, causing the tree to be smaller if the sample size is also small.

Self-regularization makes a lot of sense from a machine learning standpoint. If you have little data, you should use simpler models. So much so that both models in the preceding image have pretty decent predictive performance, as they are each optimized for the sample size they have. However, if you use these models to compute the CATE $\hat{\tau} = \mu_1(X) - \mu_0(X)$, the nonlinearity of $\mu_0(X)$ minus the linearity of $\mu_1(X)$ will result in a nonlinear CATE (dashed line minus the solid line), which is wrong, since the CATE is constant and equal to 1 in this case. You can see this happening in the second plot in the preceding image.

What happens here is that the model for the untreated can pick up the nonlinearity, but the model for the treated cannot, because it is regularized to deal with a small sample size. Of course, you could use less regularization on that model, but then it runs into the risk of overfitting. Seems like you are caught between a rock and a hard place here. How can you deal with this? Here is where the X-learner comes in.

SEE ALSO

The issue outlined here is further explored in the paper “Meta-learners for Estimating Heterogeneous Treatment Effects using Machine Learning,” by Kunzel et al.

X-Learner

The X-learner is significantly more complex to explain than the previous learner, but its implementation is quite simple, so don't worry if you don't understand it at first. The X-learner has two stages and a propensity score model. The first one is identical to the T-learner. First, you split the samples into treated and untreated and fit a model for each group:

$$\hat{\mu}_0(X) \approx E[Y|T = 0, X]$$

$$\hat{\mu}_1(X) \approx E[Y|T = 1, X]$$

Now, things start to take a turn. For the second stage, you'll first need to impute the missing potential outcomes, using the models you've fitted earlier:

$$\hat{\tau}(X, T = 0) = \hat{\mu}_1(X, T = 0) - Y_{T=0}$$

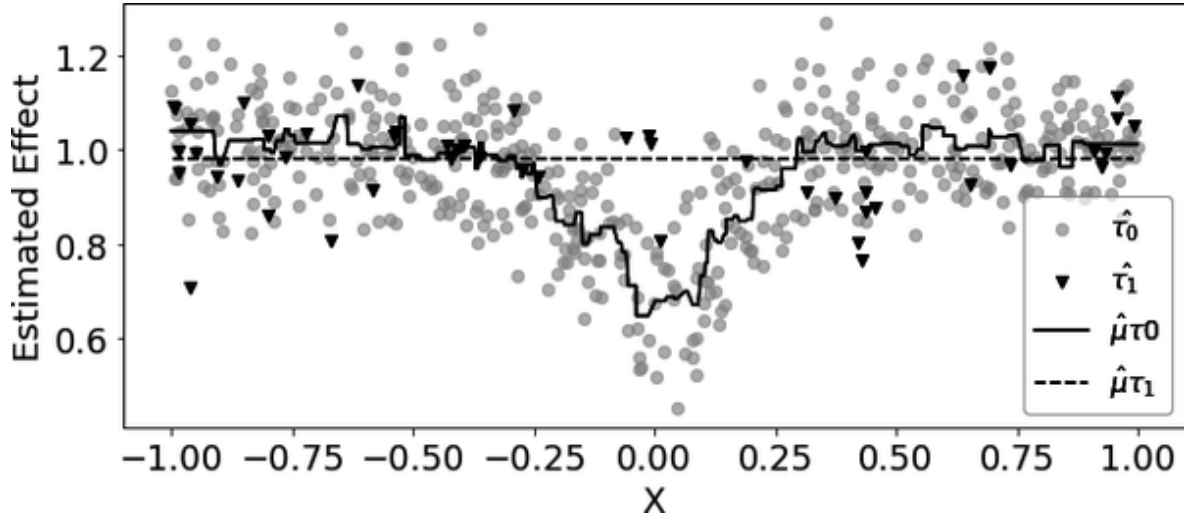
$$\hat{\tau}(X, T = 1) = Y_{T=1} - \hat{\mu}_0(X, T = 1)$$

Then, you'll fit two more models to predict those estimated effects. The idea is that this second stage of models will approximate the CATE on the control and treated populations:

$$\hat{\mu}(X)_{\tau_0} \approx E[\tau(X)|T = 0]$$

$$\hat{\mu}(X)_{\tau_1} \approx E[\tau(X)|T = 1]$$

In the illustrative data from before, the $\hat{\tau}(X, T = 0)$ and $\hat{\tau}(X, T = 1)$ are the data points in the second plot. In the following image, I'm reproducing that same data, alongside the predictive models models, $\hat{\mu}(X)_{\tau_0}$ and $\hat{\mu}(X)_{\tau_1}$. Notice that even though you have more control data, $\hat{\tau}(X, T = 0)$ is *wrong*. That is because it was constructed using $\widehat{\mu}_1$, which was fitted in a very small sample. Consequently, because $\hat{\tau}(X, T = 0)$ is wrong, $\hat{\mu}(X)_{\tau_0}$ will also be misleading. In contrast, $\hat{\mu}(X)_{\tau_1}$ will probably be correct, since $\hat{\tau}(X, T = 1)$ is also correct, as it was generated using the $\widehat{\mu}_0$ model:



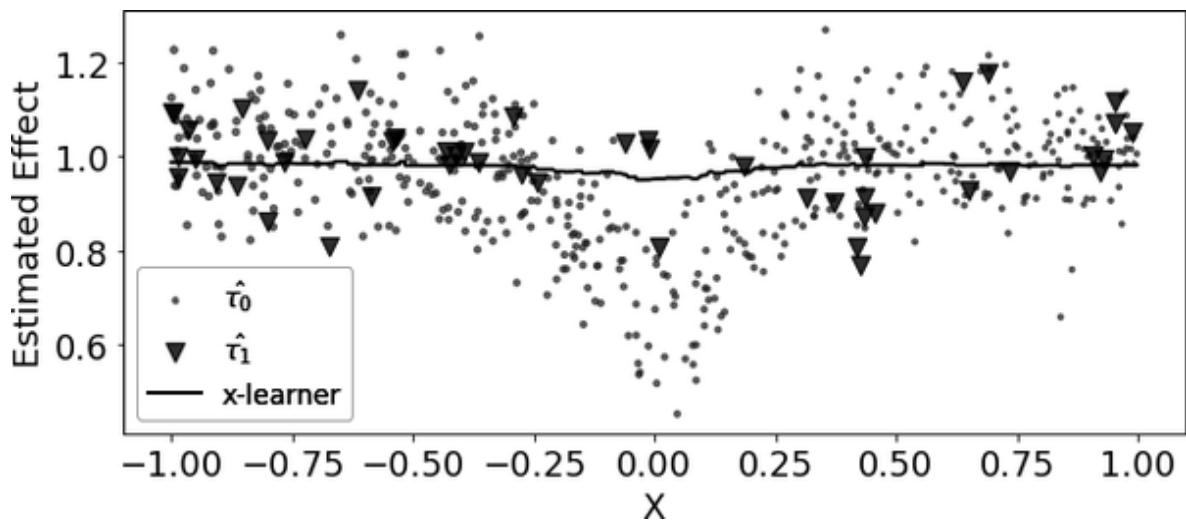
In summary, you have one model that is wrong because you've imputed the treatment effects incorrectly and another model that is correct because you've correctly imputed those values. Now, you need a way to combine the two in a way

that gives more weight to the correct model. For that, you can use a propensity score model. With it, you can combine the two second stage models as follows:

$$\widehat{\tau(x)} = \widehat{\mu}(X)_{\tau_0} \hat{e}(x) + \widehat{\mu}(X)_{\tau_1} (1 - \hat{e}(x))$$

In this example, since there are very few treated units, $\hat{e}(x)$ is very small, which gives a very small weight to the wrong CATE model, $\widehat{\mu}(X)_{\tau_0}$. In contrast, $1 - \hat{e}(x)$ is close to 1, so you will give more weight to the correct CATE model $\widehat{\mu}(X)_{\tau_1}$. More generally, this weighted average using the propensity score will favor the treatment effect estimates that were obtained from the $\widehat{\mu}_t$ model that was trained using more data.

The following image shows the estimated CATE given by the X-learner, as well as the weights assigned to each data point. Notice how it practically discards the wrong data:



As you can see, compared to the T-learner, the X-learner does a much better job in correcting the wrong CATE estimated at the nonlinearity. In general the X-learner performs better when a treatment group is much larger than the other.

I know this might be a lot to digest, but hopefully it will be clearer when you look at the code. [Figure 7-2](#) summarizes this learner.

Another thing you can try is the *domain adaptation learner*. It is the X-learner, but using the propensity score model to estimate $\hat{\mu}_t(X)$ with weights set to $1/\hat{P}(T = t)$.

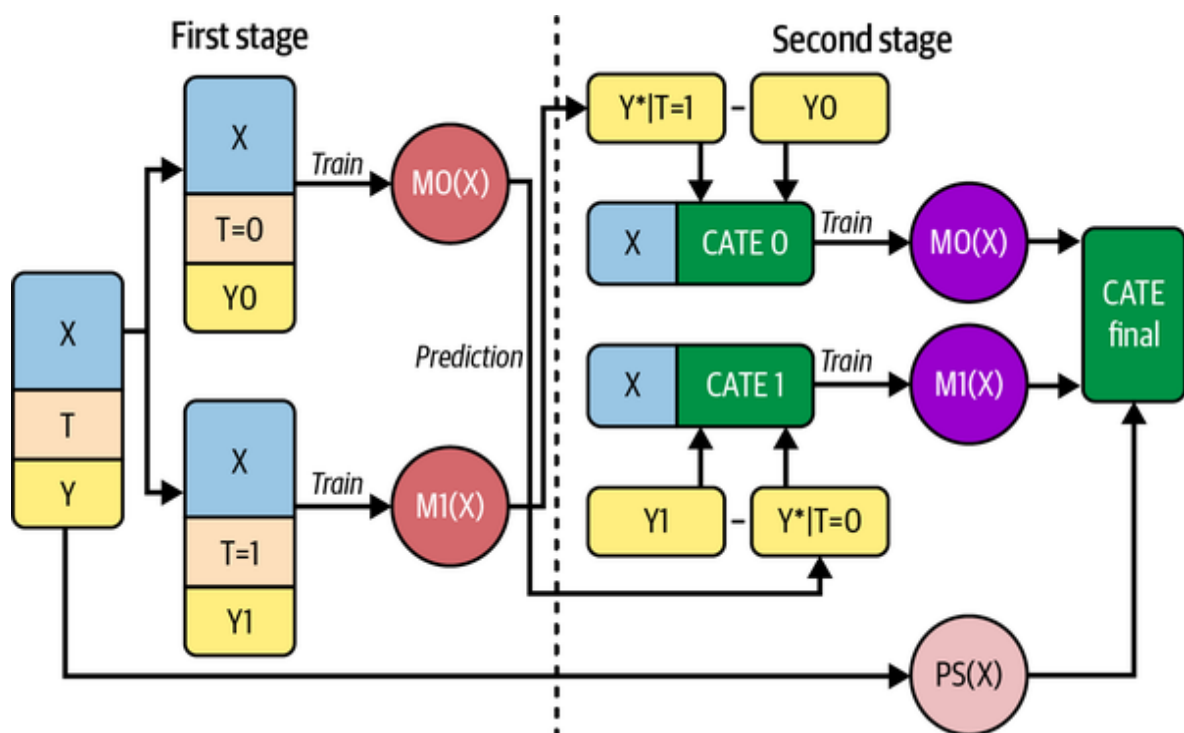


Figure 7-2. An X-learner trains two stages of ML models plus a propensity score model; at prediction time, it uses only the models from the second stage and the propensity score model

Let's see how to code all of this. Here, you have the first stage, which is exactly the same as the T-learner. If you plan on using the propensity score for domain adaptation, you need to reweight the training sample by $1/P(T = t)$, so now is also the time to fit that propensity score:

```
In [5]: from sklearn.linear_model import LogisticRegression
        from lightgbm import LGBMRegressor

        # propensity score model
        ps_model = LogisticRegression(penalty='l1')
        ps_model.fit(train[X], train[T])

        # first stage models
        train_t0 = train.query(f"{T}==0")
        train_t1 = train.query(f"{T}==1")

        m0 = LGBMRegressor()
        m1 = LGBMRegressor()

        np.random.seed(123)

        m0.fit(train_t0[X], train_t0[y],
               sample_weight=1/ps_model.predict(train_t0[X]))

        m1.fit(train_t1[X], train_t1[y],
               sample_weight=1/ps_model.predict(train_t1[X]))
```


Next, you need to predict the treatment effect and fit the second stage models on those predicted effects:

```
In [6]: # second stage
        tau_hat_0 = m1.predict(train_t0[X]) - t
        tau_hat_1 = train_t1[y] - m0.predict(tr

        m_tau_0 = LGBMRegressor()
        m_tau_1 = LGBMRegressor()

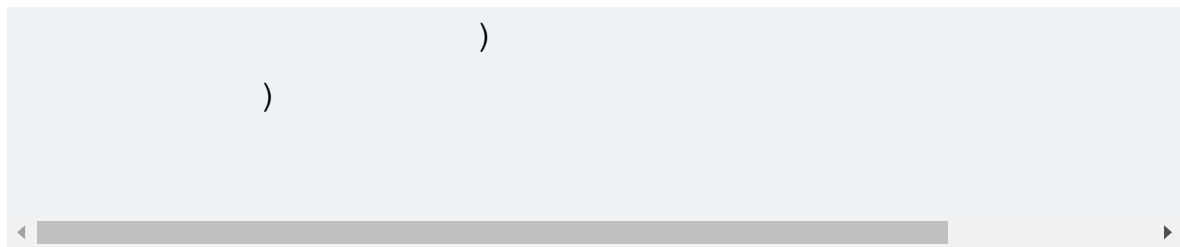
        np.random.seed(123)

        m_tau_0.fit(train_t0[X], tau_hat_0)
        m_tau_1.fit(train_t1[X], tau_hat_1);
```

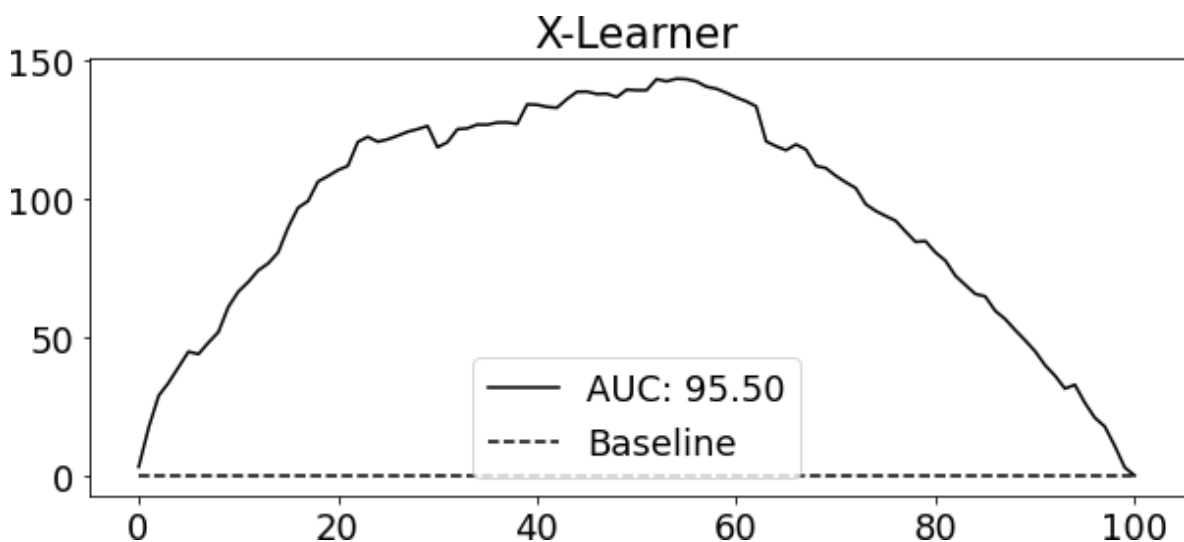
Finally, once you have all of that, you can combine the predictions from the second-stage models using the propensity score model to obtain the CATE. All of which can be estimated on the test set:

```
In [7]: # estimate the CATE
        ps_test = ps_model.predict_proba(test[X

        x_cate_test = test.assign(
            cate=(ps_test*m_tau_0.predict(test
                (1-ps_test)*m_tau_1.predict(t
```



Let's see how the X-learner does in terms of the cumulative gain. In this data set, the treatment and control are almost the same size, so don't expect a huge difference. The issue the X-learner tries to correct probably does not manifest here:



As expected, the X-learner performance is not very different from what you got with the T-learner. In fact, it slightly underperforms it, in terms of the area under the curve. Keep in mind that the quality of these learners is situation-dependent. Like I said earlier, in this specific data, both the treatment and control have a decent enough sample size so as to not run into the type of problem that the X-learner tries

to solve. This might explain the similar performance between the two models.

Metalearners for Continuous Treatments

As always, when the treatment is continuous, things can get a bit complicated. It is no different with the metalearners. As a running example, let's use the data from the previous chapter. Recall that it has three years' worth of data from a chain of restaurants. The chain randomized discounts on six of its restaurants and it now wants to know which are the best days to give more discounts. To answer this question, you need to understand on which days customers are more sensitive to discounts (more sensitive to prices). If the restaurant chain can learn this, they will be better equipped to decide when to give more or fewer discounts.

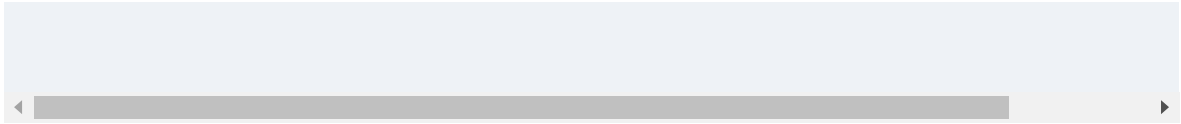
As you can see, this is a problem where you need to estimate the CATE. If you manage to do so, the company can use your CATE predictions to decide on a discount policy—the higher the predicted CATE, the more customers are sensitive to discounts, so the higher the discounts should be:

```
In [8]: data_cont = pd.read_csv("../data/discour  
      data_cont.head()
```

	rest_id	day	month
0	0	2016-01-01	1
1	0	2016-01-02	1
2	0	2016-01-03	1
3	0	2016-01-04	1
4	0	2016-01-05	1

In this data, discounts is the treatment and sales is the outcome. You also have some engineered date features, like the month, the day of the week, if it is a holiday and so on. Since the goal here is CATE prediction, it is probably best to split your dataset into a training and a test set. Here, you can take advantage of the time dimension and use it to create those sets:

```
In [9]: train = data_cont.query("day<' 2018-01-01")
        test = data_cont.query("day>=' 2018-01-01")
```



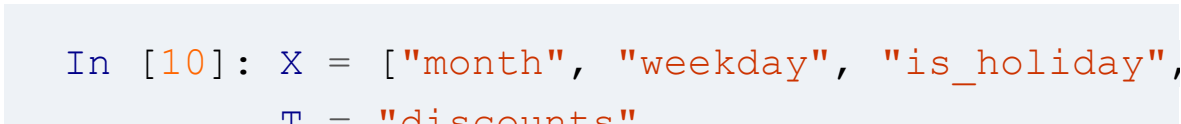
Now that you are familiar with the data, let's see which of the metalearners can deal with this continuous treatment.

S-Learner

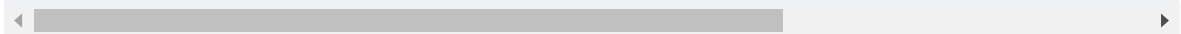
The first learner you should try is the S-learner. This is the simplest learner there is. You'll use a single (hence the S) machine learning model $\hat{\mu}_s$ to estimate:

$$\mu(x) = E[Y|T, X]$$

To do so, you will include the treatment as a feature in the model that tries to predict the outcome Y . That's pretty much it:



```
In [10]: X = ["month", "weekday", "is_holiday",  
            T = "discounts"  
            y = "sales"  
  
np.random.seed(123)  
s_learner = LGBMRegressor()  
s_learner.fit(train[X+[T]], train[y]);
```



But this model does not output a treatment effect directly. Rather, it outputs counterfactual predictions. That is, it can make predictions under different treatment regimes. If the

treatment were binary, this model would still work and the difference in predictions between the test and control will be the CATE estimate:

$$\hat{\tau}(x)_i = M_s(X_i, T = 1) - M_s(X_i, T = 0)$$

[Figure 7-3](#) contains a diagram that explain what it would look like.

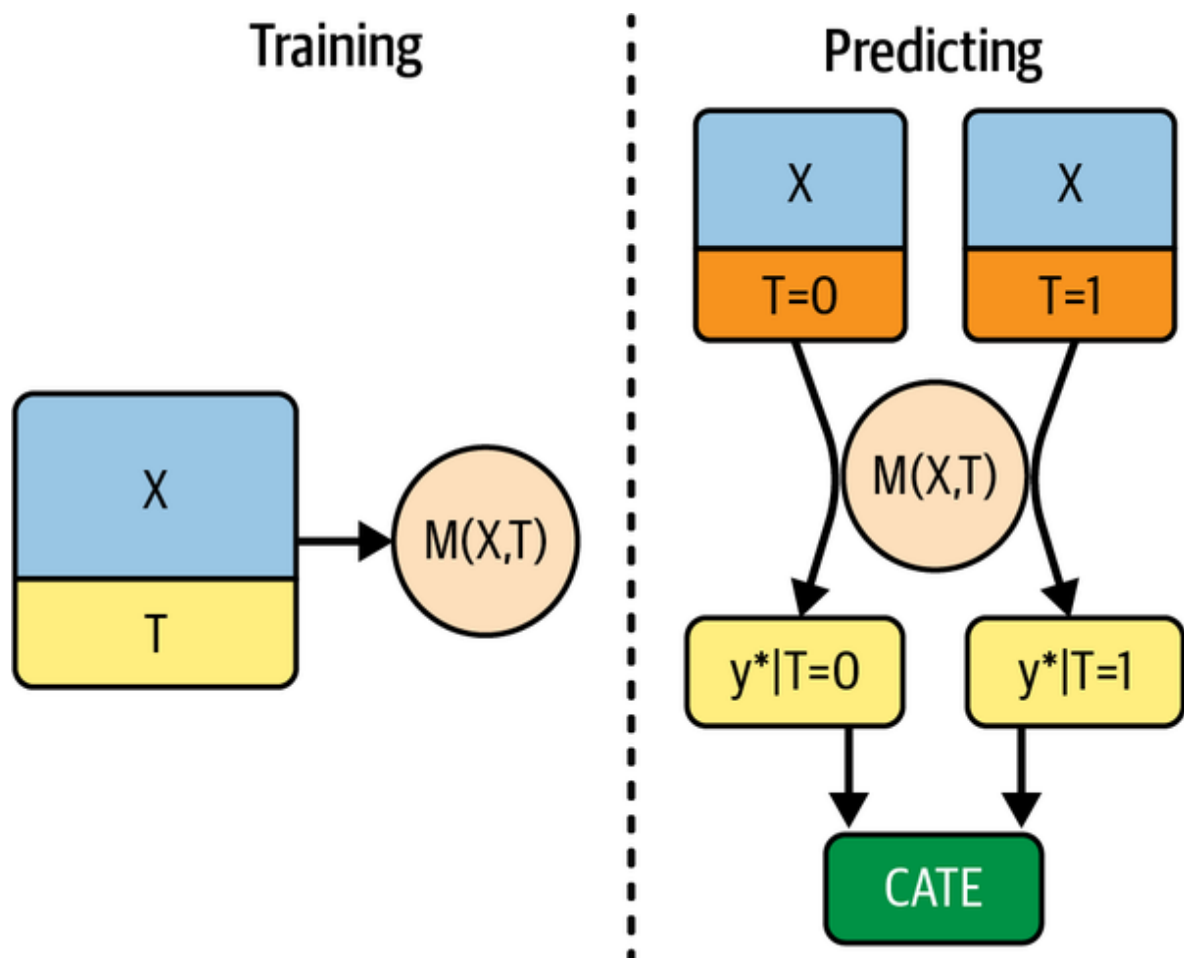


Figure 7-3. An S-learner is simply an ML model that has the treatment as one of its features

In the continuous case, you have to do a bit of extra work. First, you need to define a grid of treatments. In the example, the discounts go from zero to about 40%, so you can try a

```
[0, 10, 20, 30, 40] grid.
```

Next, you need to expand the data you want to make predictions on so that each line gets one copy for each treatment value in the grid. The easiest way I can find to do that is to cross-join a data frame with the grid values into the data where I want to make predictions—the test set. In pandas, you can do a cross-join by using a constant key. This will replicate each line in the original data, changing only the treatment value. Finally, you can use your fitted S-learner to make counterfactual predictions in this expanded data. Here is a simple piece of code to do all of that:

```
In [11]: t_grid = pd.DataFrame(dict(key=1,
                                     discounts=r

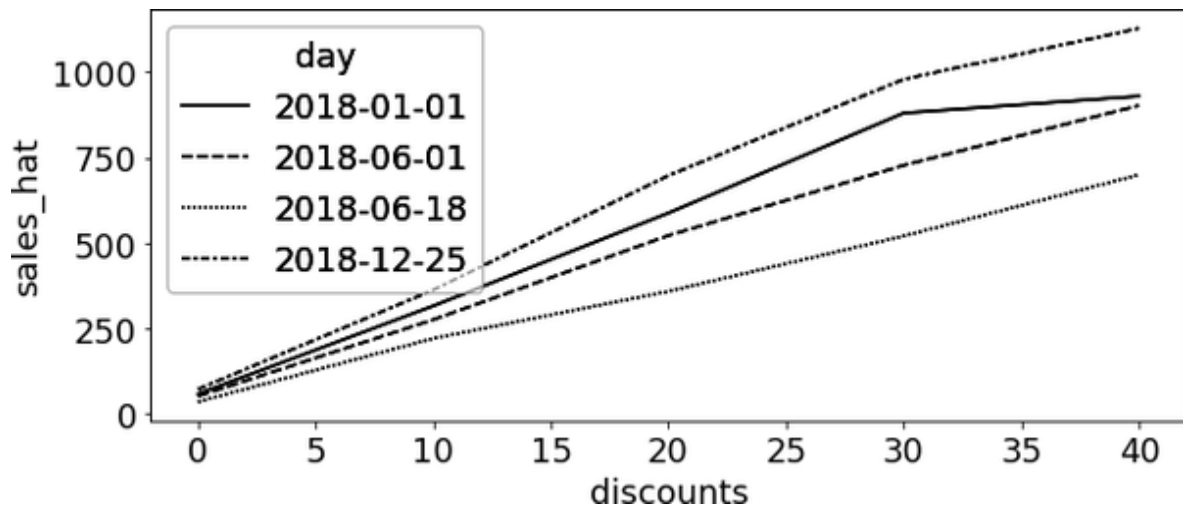
test_cf = (test
            .drop(columns=["discounts"])
            .assign(key=1)
            .merge(t_grid)
            # make predictions after ex
            .assign(sales_hat = lambda

test_cf.head(8)
```

	rest_id	day	month
0	0	2018-01-01	1
1	0	2018-01-01	1
2	0	2018-01-01	1
3	0	2018-01-01	1
4	0	2018-01-01	1
5	0	2018-01-02	1
6	0	2018-01-02	1
7	0	2018-01-02	1

In the previous step, you’ve essentially estimated a coarse version of the treatment response function $Y(t)$ for each unit. You can even plot this curve for a handful of units (days, in our case) to see what they look like. In the following plot, you can see that the estimated response function for 2018-12-25—that is, Christmas—is steeper than the one for a day like

2018-06-18. This means that your model learned that customers are more sensitive to discounts on Christmas, compared to that particular day in June:



Whether those counterfactual predictions are correct is a whole other issue. To evaluate this model, you first need to realize that you still don't have a CATE prediction. This means that the evaluation methods you learned in [Chapter 6](#) can't be used here. In order to get a CATE prediction, you have to somehow summarize the unit level curves into a single number that represents the treatment effect.

Surprisingly—or not that much—linear regression is a good way of doing that. Simply put, you can run a regression for each unit and extract the slope parameter on the treatment as your CATE estimate.

Since all you care about is the slope parameter, you can do this much more efficiently, using the formula for the slope of

the single variable linear regression:

$$\hat{\beta} = Cov(t, y) / Var(t)$$

Let's see the code to do that. First, I'm defining a function that summarizes each individual curve into a slope parameter.

Then, I'm grouping the expanded test data by the restaurant ID and day and applying the slope function to each of those units. This gives me a pandas series, with indexes

`rest id` and `day`. I'm naming this series `cate`. Finally, I'm joining the series into the original

test set (not the expanded one) to get a CATE prediction for each day and restaurant in the testing set:

```
In [12]: from toolz import curry

@curry
def linear_effect(df, y, t):
    return np.cov(df[y], df[t])[0, 1]

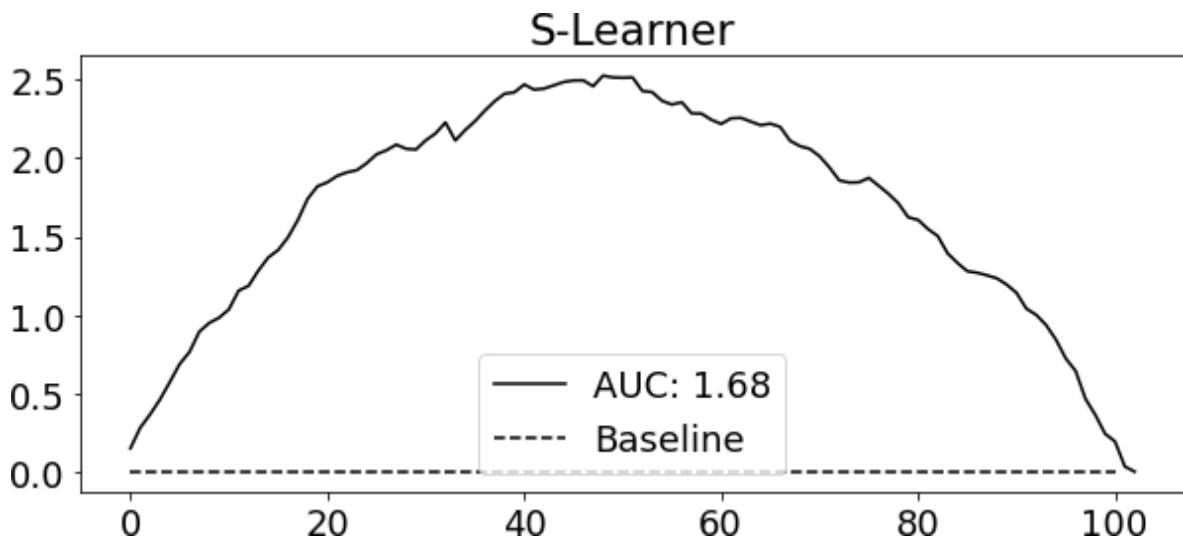
cate = (test_cf
        .groupby(["rest_id", "day"])
        .apply(linear_effect(t="discount"))
        .rename("cate"))

test_s_learner_pred = test.set_index(
```

```
test_s_learner_pred.head()
```

		month	weekday
rest_id	day		
0	2018-01-01	1	0
	2018-01-02	1	1
	2018-01-03	1	2
	2018-01-04	1	3
	2018-01-05	1	4

Now that you have a CATE prediction, you can use the methods you learned from the previous chapter to validate your model. Here, let's stick with the cumulative gain:

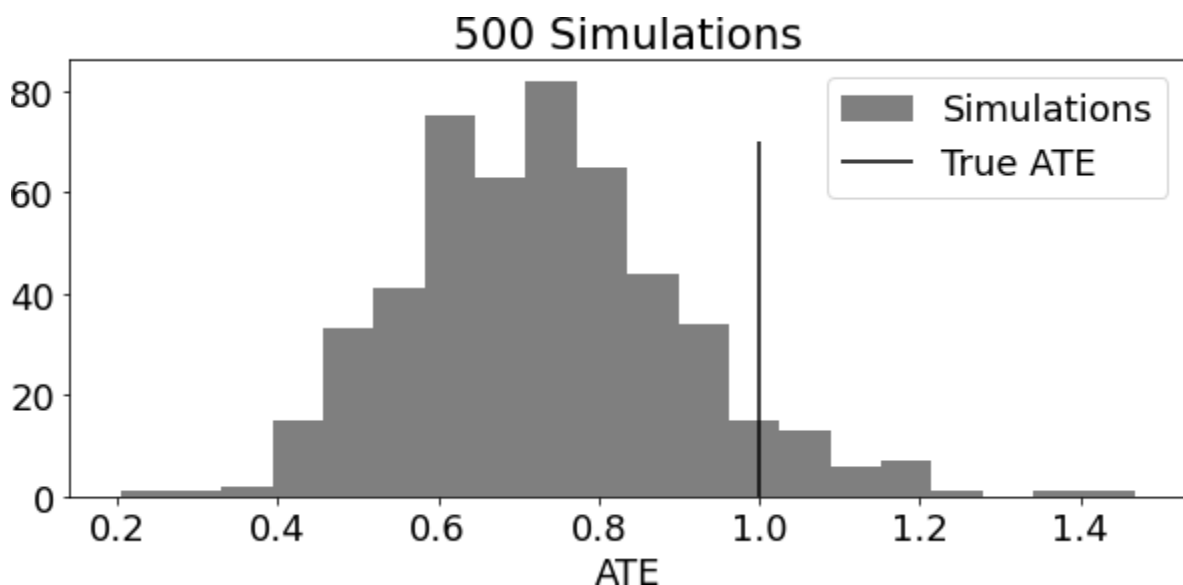


As you can see from the cumulative gain, the S-learner, although simple, can perform OK on this dataset. Again, keep in mind that this performance is highly particular to this dataset. This is a particularly easy dataset, as you have lots of random data, which you can use even to train your learner. In practice, I find that the S-learner is a good first bet for any causal problem, mostly due to its simplicity. It also tends to perform OK, even if it doesn't have random data to train. Moreover, the S-learner supports both binary and continuous treatment, making it an excellent default choice.

The major disadvantage of the S-learner is that it tends to bias the treatment effect toward zero. Since the S-learner employs what is usually a regularized machine learning model, that regularization can restrict the estimated treatment effect.

The following plot replicates a result from the paper, "Double/Debiased/Neyman Machine Learning of Treatment

Effects,” by Chernozhukov et al. To make this plot, I simulated data with 20 covariates and a binary treatment with a true ATE of 1. I then tried to estimate that ATE using an S-learner. I repeated this simulation and estimation steps 500 times and plotted the distribution of the estimated ATE alongside the true ATE:



You can see that the distribution of estimated ATEs is concentrated to the left of the true ATE, being biased toward zero. In other words, the true causal effect is frequently bigger than the estimated one.

Even worse, if the treatment is very weak relative to the impact other covariates play in explaining the outcome, the S-learner can discard the treatment variable completely. Notice that this is highly related to the chosen ML model you employ. The greater the regularization, the greater the

problem. A way around this, proposed in the same paper by Chernozhukov et al., is Double/Debiased Machine Learning, or the R-learner.

Double/Debiased Machine Learning

Double/Debiased ML or the R-learner can be seen as a buffed version of the Frisch-Waugh-Lovell theorem. The idea is very simple—use ML models when constructing the outcome and treatment residuals:

$$Y_i - \hat{\mu}_y(X_i) = \tau \cdot (T_i - \hat{\mu}_t(X_i)) + \epsilon_i$$

where $\hat{\mu}_y(X_i)$ is estimating $E[Y|X]$ and $\hat{\mu}_t(X_i)$ is estimating $E[T|X]$.

Since the ML models can be super flexible, they are better suited to capture interactions and nonlinearities when estimating the Y and T residuals while still maintaining an FWL-style orthogonalization. This means you don't have to make any parametric assumption about the relationship between the covariates X and the outcome Y nor between the covariates and the treatment in order to get the correct treatment effect. Provided you don't have unobserved confounders, you can recover the ATE with the following orthogonalization procedure:

1. Estimate the outcome Y with features X using a flexible ML regression model μ_y .
2. Estimate the treatment T with features X using a flexible ML regression model μ_t .
3. Obtain the residuals $\tilde{Y} = Y - \mu_y(X)$ and $\tilde{T} = T - \mu_t(X)$.
4. Regress the residuals of the outcome on the residuals of the treatment $\tilde{Y} = \alpha + \tau\tilde{T}$, where τ is the causal parameter ATE, which you can estimate, for example, with OLS.

The power you gain with ML is flexibility. ML is so powerful that it can capture complicated functional forms in the nuisance relationships. But that flexibility is also troublesome, because it means you now have to take into account the possibility of overfitting. The paper by Chernozhukov et al. has a much more in-depth and rigorous explanation about how overfitting can be troublesome and I definitely recommend you check it out. But here, I'll go on with a more intuition-based explanation.

To see the issue, suppose that your μ_y model is overfitting. The result is that the residual $\tilde{Y}$ will be smaller than it should be. It also means that μ_y is capturing more than only the relationship between X and Y . Part of that something more is the relationship between T and Y , and if μ_y is capturing

some of that, the residual regression will be biased toward zero. In other words, μ_y is capturing the causal relationship and not leaving it to the final residual regression.

Now, to see the problem in overfitting μ_t , notice that it will explain more of the variance in T than it should. As a result, the treatment residual will have less variance than it should. If there is less variance in the treatment, the variance of the final estimator will be high. It is as if the treatment is the same for almost everyone, or if the positivity assumption was violated. If everyone has almost the same treatment level, it becomes very difficult to estimate what would happen under different treatment.

Those are the problems you have when using ML models. But how can you work around them? The answer lies in cross predictions and out-of-fold residuals. Instead of getting the residuals in the same data used to fit the model, you'll partition your data into K folds, estimating the model in $K-1$ of those folds and getting the residuals in the fold that was left out. Repeat the same procedure K times to get the residuals for the entire dataset. With this approach, even if the model does overfit, it won't drive the residuals to zero artificially.

This looks complicated in theory but it is actually very easy to code. You can use the `cross_val_predict` function from `sklearn` to get out-of-fold predictions from any

machine learning model. Here is how you can get those residuals with just a few lines of code:

```
In [13]: from sklearn.model_selection import cross_val_predict

X = ["month", "weekday", "is_holiday",
T = "discounts"
y = "sales"

debias_m = LGBMRegressor()
denoise_m = LGBMRegressor()

t_res = train[T] - cross_val_predict(denoise_m, train[X], train[y])
y_res = train[y] - cross_val_predict(debias_m, train[X], train[y])
```

If you only cared about the ATE, you could simply regress the residual of the outcome on the residual of the treatment (just don't trust those standard errors, as they don't account for the variance in estimating the residuals):

```
In [14]: import statsmodels.api as sm

sm.OLS(y_res, t_res).fit().summary().t
```

	coef	std err	t	P> t
discounts	31.4615	0.151	208.990	0.000

But in this chapter, we are focusing on CATE. So how exactly do you get that with Double-ML?

Double-ML for CATE estimation

To get CATE predictions from your Double-ML model, you'll need a few adaptations. Essentially, you need to allow the causal parameter τ to change depending on the unit's covariates:

$$Y_i = \hat{\mu}_y(X_i) + \tau(X_i)(T_i - \hat{\mu}_t(X)) + \hat{\epsilon}_i$$

where $\hat{\mu}_y$ and $\hat{\mu}_t$ are models that, respectively, predict the outcome and treatment from the features X . If you rearrange the terms, you can isolate the error:

$$\hat{\epsilon}_i = (Y_i - \hat{\mu}_y(X_i)) - \tau(X_i)(T_i - \hat{\mu}_t(X))$$

This is nothing short of awesome, because now you can call this a *causal loss function*. Which means that, if you minimize the square of this loss, you'll be estimating the expected value of $\tau(X_i)$, which is the CATE you wanted:

$$\hat{L}_n(\tau(x)) = \frac{1}{n} \sum_{i=1}^n \left((Y_i - \hat{M}_y(X_i)) - \tau(X_i)(T_i - \hat{M}_t(X)) \right)^2$$

This loss is also called the *R-Loss*, since it's what the R-learner minimizes. OK, but how do you minimize this loss function? There are multiple ways, actually, but here you'll see the simplest one. First, to declutter the technical notation, let's rewrite the loss function using the residualized version of treatment and outcome:

$$\hat{L}_n(\tau(x)) = \frac{1}{n} \sum_{i=1}^n \left(\tilde{Y}_i - \tau(X_i) \tilde{T}_i \right)^2$$

Finally, you can do some algebraic parkour to take $\tilde{T}_i$ out of the parentheses and isolate $\tau(X_i)$ in the square part of the loss function:

$$\hat{L}_n(\tau(x)) = \frac{1}{n} \sum_{i=1}^n \tilde{T}_i^2 \left(\frac{\tilde{Y}_i}{\tilde{T}_i} - \tau(X_i) \right)^2$$

Minimizing the preceding loss is equivalent to minimizing what is inside the parentheses, but weighting each term by $\tilde{T}_i^2$. Any predictive ML model can do that.

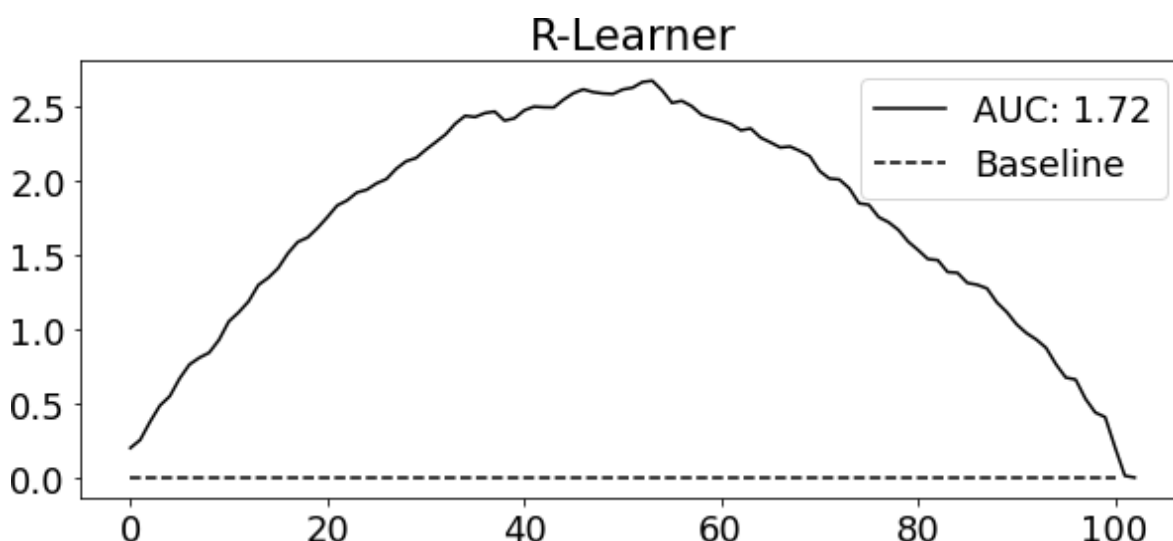
But wait a minute! You saw this already! This is the transformed target you used to compute the mean square error in [Chapter 6](#)! Indeed it is. Then, I asked you to trust my word for it, but now I offer you the reason why it works. Again, coding this up is very simple:

```
In [15]: y_star = y_res/t_res  
         w = t_res**2
```

```
cate_model = LGBMRegressor().fit(train_data)

test_r_learner_pred = test.assign(cate
```

What I really like about this learner is that it directly outputs CATE estimates. There is no need for all those extra steps you had to take with the S-learner. Also, as you can see in the following plot, it does a pretty decent job in terms of ordering the CATE, as measured by the cumulative gain:



In this example, Double/Debiased-ML has a pretty similar performance to the S-learner. That is probably because the treatment is strong enough so that the ML model in S-learner assigns high importance to it. Also, the treatment is randomized, which means that the μ_t model in Double-ML is not doing anything really. So, in order to get a better

understanding of the true power of Double-ML, let's go through a more illustrative example.

Visual intuition for Double-ML

Consider the following simulated data. In it, you have two covariates: x_c is a confounder and x_h is not. Also, x_h drives effect heterogeneity. There are only three values for x_h : 1, 2, and 3. The CATE for each of them is 2, 3, and 4, respectively, since the treatment effect is given by $t + tx_h$. Also, since x_h is uniformly distributed, the ATE is just a simple average of the CATEs—that is, 3. Finally, notice how the confounder x_c affects both treatment and outcome nonlinearly:

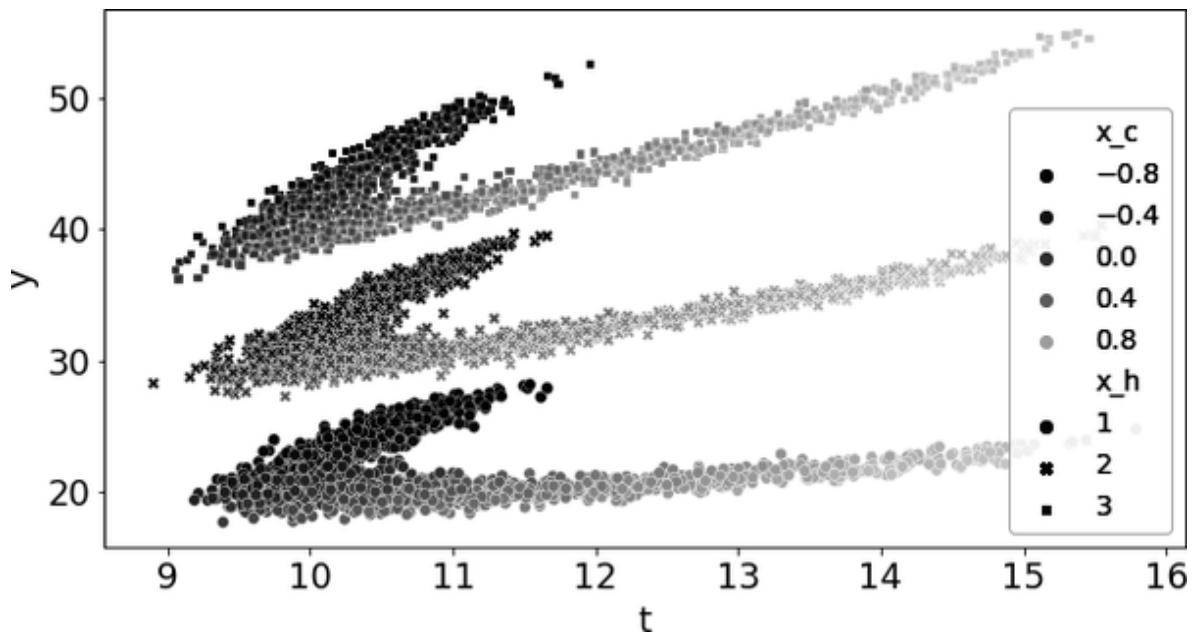
```
In [16]: np.random.seed(123)
         n = 5000

         x_h = np.random.randint(1, 4, n)
         x_c = np.random.uniform(-1, 1, n)

         t = np.random.normal(10 + 1*x_c + 3*x_h, 1, n)
         y = np.random.normal(t + x_h*t - 5*x_c, 1, n)

         df_sim = pd.DataFrame(dict(x_h=x_h, x_c=x_c, t=t, y=y))
```

Here is a plot for this data. Each blob of points is a group defined by x_h . The color coding represents the value of the confounder x_c . Notice the nonlinear shape in it:



Now, let's see how Double-ML processes this data. First, let's get the residuals $\tilde{T}$ and $\tilde{Y}$. Since you don't have a lot of data here, constrain your ML models to have trees with

`max_depth=3`

. I'm including only x_c in the debiasing model, since that is the only confounder. The denoise model has both covariates, as both cause the outcome and including them will reduce noise:

```
In [17]: debias_m = LGBMRegressor(max_depth=3)
         denoise_m = LGBMRegressor(max_depth=3)

         t_res = cross_val_predict(debias_m, df
                                   cv=10)
```

```

y_res = cross_val_predict(denoise_m, c
                           cv=10)

df_res = df_sim.assign(
    t_res = df_sim["t"] - t_res,
    y_res = df_sim["y"] - y_res
)

```

Once you have those residuals, the confounding bias due to x_c should be gone. Even though it is nonlinear, our ML model should be able to capture that nonlinearity and get rid of all the bias. So much so that if you run a simple regression of $\tilde{Y}$ on $\tilde{T}$, it should give you the correct ATE:

```

In [18]: import statsmodels.formula.api as smf

smf.ols("y_res~t_res", data=df_res).fit()

```

```

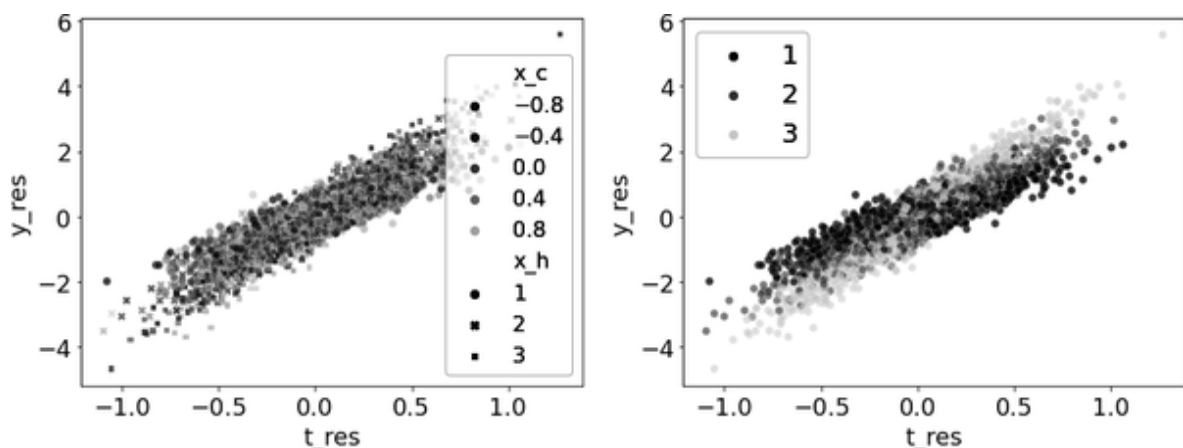
Out[18]: 3.045230146006292

```

Next, let's turn our attention to CATE estimation. The left plot in the following figure shows the relationship between the

residuals and color codes each point by the confounder x_c . Notice that there are no patterns in the color of this plot. This shows that all confounding due to x_c was removed. The data looks as if the treatment was randomly assigned.

The next plot color codes the same relationship by the x_h , the feature that drives treatment heterogeneity. The darkest points ($x_h = 1$) seem to be less sensitive to the treatment, as shown by the lower slope. In contrast, the lighter ones ($x_h = 3$) seem to be more sensitive to the treatment. Looking at this plot, can you think of a way to extract those sensitivities?



To answer this question, notice that both residuals are centered around zero. This means that the lines that dictate the slope of all the groups defined by x_h should cross zero. Now, recall that the slope of a line can be estimated from two points as $\Delta y / \Delta t$. But, since the intercept of this line should be zero, this simplifies to y/t . Hence, you can see the Y^*

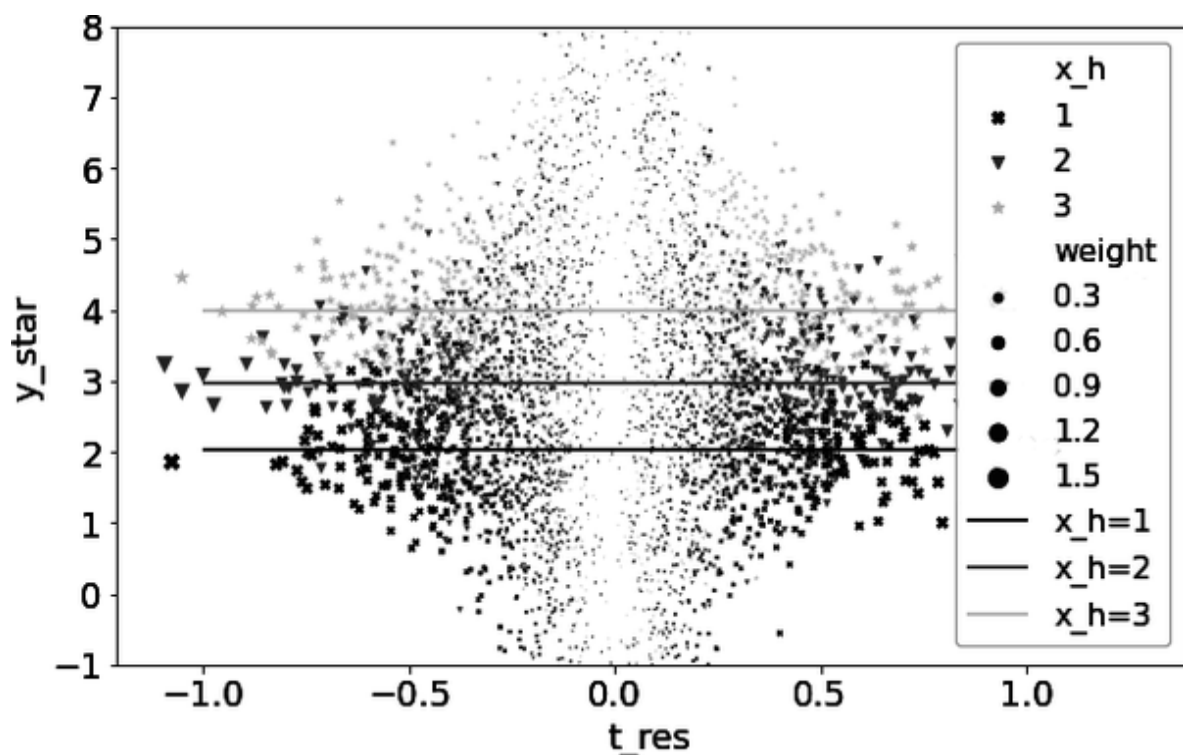
target from Double-ML as the slope of the line that goes through the point and has zero as its intercept.

But there is a catch. Both $\tilde{T}$ and $\tilde{Y}$ have a mean close to zero. You know what happens when you divide by a number close to zero? That's right, it can be very unstable, giving you tremendous amounts of noise. Here is where the weights $\tilde{T}^2$ come into play. By giving more importance to points with high values of $\tilde{T}$, you are essentially focusing on the region where the variance is low. To see that this works, you can compute the average of Y^* , weighted by $\tilde{T}^2$, for each value of x_h . This will get you pretty close to the true CATE of 2, 3, and 4, for $x_h = 1, 2, 3$, respectively:

```
In [19]: df_star = df_res.assign(  
        y_star = df_res["y_res"]/df_res["t_res"],  
        weight = df_res["t_res"]**2,  
    )  
  
    for x in range(1, 4):  
        cate = np.average(df_star.query(f"x_h={x}"),  
                          weights=df_star.weight,  
                          axis=0)  
  
        print(f"CATE x_h={x}", cate)
```

```
Out[19]: CATE x_h=1 2.019759619990067
          CATE x_h=2 2.974967932350952
          CATE x_h=3 3.9962382855476957
```

You can also see what I'm talking about in the plot of $\tilde{Y}^*$ by $\tilde{T}$. Here, I'm again color coding by x_h , but now I'm adding weights equal to $\tilde{T}^2$. I've also included the average estimated CATE for each group as a horizontal line:



I like this plot because it clearly shows the role of the weights.

The variance of $\tilde{Y}^*$ increases a lot as you approach the center of the plot. You can't see it because I've limited the range of the y-axis, but you actually have points that go all

the way to both $-2,000$ and $2,000$! Fortunately, those are all close to $\tilde{T} = 0$, so they have very small weights. And now you know on a more intuitive level what is going on with Double-ML.

TREE-BASED LEARNERS AND NEURAL NET LEARNERS

This chapter doesn't intend to be an exhaustive list of all the meta-learners there currently are. I've only included the ones that I personally find most useful. However, beyond the four learners presented here, there are some others that are worth mentioning.

First, Susan Athey and Stefan Wager did a lot of pioneering work on effect heterogeneity using modified decision trees. You can find tree-based CATE learners in causal inference libraries such as `econml` and `causalml`. I did not include them in this chapter because, at the time of this writing, I've never managed to use them successfully. Mostly because the implementations currently available are in pure Python, which makes them quite slow to fit on large datasets. I do expect that sometime soon a faster implementation will arise, making tree-based learners an interesting option to try out. If you want to learn more about tree-based learners, I suggest the documentation of the causal inference packages that implement them. There is also a fantastic online series of videos from Stanford Business School, by Athey and Wager, called *Machine Learning & Causal Inference: A Short Course*.

Second, there are neural network-based algorithms you can try. However, I think those are still in their infancy and the amount of complexity they bring is not worth the potential gain. At least not yet. Still, if you want to venture yourself in

this literature, I recommend you check the papers “Nonparametric Estimation of Heterogeneous Treatment Effects: From Theory to Learning Algorithms”, by Curth and Schaar, and “Learning Representations for Counterfactual Inference,” by Shalit et al.

Key Ideas

This chapter expands on the idea of learning group-level treatment effects $\tau(x_i)$. Instead of just interacting the treatment variable with the covariates X in a regression model, you learned how to repurpose generic machine learning models for conditional average treatment effect (CATE) estimation: the so-called metalearners. Specifically, you learned about four meta-learners, two that work only with categorical treatments and two that work with any type of treatment.

First, the T-learner fits a machine learning model to predict the Y for each treatment T . Then, the resulting outcome models $\widehat{\mu}_t$ can be used to estimate the treatment effect. For instance, in the case of a binary treatment:

$$\hat{\tau}(X_i) = \widehat{\mu}_1(X_i) - \widehat{\mu}_0(X_i)$$

the T-learner works fine if you have lots of observations for all treatment levels. Otherwise, the model estimated in a

small dataset can suffer from regularization bias. The next learner you saw, the X-learner, tried to address this issue by using a propensity score model to lower the importance of any $\widehat{\mu}_t$ trained on a small sample.

To handle continuous treatment, you learned about the S-learner, which simply estimates $E[Y|T, X]$. That is, it predicts the outcome with the treatment included as a feature. This model can be used to make counterfactual predictions of Y_t , given a grid of treatment values. This results in a unit-specific coarse treatment response function, which later needs to be summarized into a single slope parameter.

Last, but not least, you learned about Double-ML. The idea was to use generic ML models and out-of-fold prediction to get treatment and outcome residuals, $T - E[T|X]$ and $Y - E[Y|X]$, respectively. This can be understood as a buffed version of FWL orthogonalization. Once you have those residuals—call them $\tilde{T}$ and $\tilde{Y}$ —you could construct a target that approximates $\tau(x_i)$:

$$Y^* = \tilde{Y} / \tilde{T}$$

Using any ML model to predict that target while also using weights $\tilde{T}^2$ resulted in a ML model that could output CATE predictions directly.

Finally, it's worth remembering that all these methods rely on the unconfoundedness assumptions. It doesn't matter how cool-sounding the algorithm you are trying to use for CATE estimation is; for them to be able to remove bias, you need to have in your data all the relevant confounders. Specifically, unconfoundedness allows you to interpret rates of change on the conditional expectation as if it was the slope of the treatment response function:

$$\frac{\partial}{\partial t} E[Y(t)|X] = \frac{\partial}{\partial t} E[Y|T = t, X]$$